Speech Notes

A Speech-to-Text Meeting Notes Prototype with Word Error Rate Evaluation

Course code: AAI-010 | Speech AI

Project team: [Name 1], [Name 2], [Name 3]

Student IDs: [add]

Supervisor: [add]

Submission date: [add]

Repository: github.com/tripathi-yash/speech-notes

Live application: huggingface.co/spaces/Ykode/speech-notes

Declaration

We declare that this project report is our own work. Where we have used the work of others, it has been acknowledged in the text and in the reference list. [Each team member signs here.]

Acknowledgements

[Add acknowledgements: supervisor, lab staff, and any tools or data providers.] We thank the authors of Whisper, the LibriSpeech corpus, the Gradio project, and the open-source communities behind ffmpeg, PyTorch and Hugging Face Spaces.

Abstract

Students often need recorded lectures and meetings converted into searchable text, but manual transcription is slow. This project builds Speech Notes, a prototype that converts short speech clips into text with the pretrained Whisper model, cleans the output into readable lines, and measures its accuracy against a manually checked reference using Word Error Rate (WER). The system has two front ends that share the same core code: a command-line pipeline that scores a folder of clips and produces a PDF report, and a web application built with Gradio and deployed on Hugging Face Spaces. We evaluated the system on LibriSpeech clips. A 59-word clip scored 3.39% WER after an abbreviation fix in the scoring, and a 110-word clip scored 10.00% WER. Error analysis shows that most mistakes involve proper names, unusual vocabulary and word boundaries, not the overall structure of the transcript. We discuss the research behind the approach, the architecture of the system and of the Whisper model, the evaluation method, the deployment, and the responsible-use constraints for recorded speech.

Keywords: automatic speech recognition, Whisper, word error rate, Levenshtein alignment, Gradio, Hugging Face Spaces, speech-to-text, error analysis.

Contents

Declaration 2

Acknowledgements 2

Abstract 2

1. Introduction 6

1.1 Background and motivation . . . 6

1.2 Problem statement . . . 6

1.3 Objectives . . . 6

1.4 Scope and contributions . . . 6

2. Research and Background 7

2.1 Automatic speech recognition: a short history . . . 7

2.2 Choosing a recogniser for this project . . . 7

2.3 Measuring recognition accuracy . . . 7

2.4 Datasets . . . 8

2.5 Responsible use of speech data . . . 8

3. Requirements Analysis 9

3.1 Functional requirements . . . 9

3.2 Non-functional requirements . . . 9

3.3 Data requirements . . . 9

4. System Architecture 10

4.1 Overview . . . 10

4.2 Components . . . 10

4.3 Design decisions . . . 10

4.4 Evolution of the architecture . . . 11

5. The Recognition Model 12

5.1 Whisper in brief . . . 12

5.2 Stages in detail . . . 12

5.3 Model sizes . . . 12
5.4 Settings recorded with every transcript . . . 13
6. Implementation 14
6.1 Text clean-up (src/cleanup.py) . . . 14
6.2 Word error rate (src/wer.py) . . . 14
6.3 Recognition (src/transcribe.py and app.py) . . . 14
6.4 Evaluation and reporting (src/evaluate.py and report/make_report.py) . . . 14
6.5 Clip construction (src/make_clip.py) . . . 14
6.6 Packaging and dependencies . . . 15
6.7 Code quality and tests . . . 15
7. Experimental Setup 16
7.1 Data . . . 16
7.2 Hardware and software . . . 16
7.3 Procedure . . . 16
7.4 Procedure for the abbreviation change . . . 16
8. Evaluation Method 17
8.1 Error types . . . 17
8.2 Qualitative review . . . 17
8.3 Limits of the metric . . . 17
9. Results 18
9.1 Summary . . . 18
9.2 Interpretation . . . 18
9.3 Clip-level statistics . . . 18
10. Error Analysis 19
10.1 Clip 2: all 11 errors . . . 19
10.2 Summary by category . . . 19
10.3 Clip 1: the remaining errors . . . 19
10.4 What the errors tell us . . . 20

10.5 Qualitative examples 20
11. End-to-End Walkthrough: Backend to Frontend 21
11.1 Page load 21
11.2 Upload and parameters 21
11.3 Transcribe click 21
11.4 Results on screen 21
11.5 Error handling 21
11.6 Deployment path 22
12. Development Log and Issues Resolved 23
13. Responsible Use, Privacy and Licensing 24
13.1 Privacy 24
13.2 Accuracy and misuse 24
13.3 Licences 24
14. Limitations and Future Work 25
14.1 Limitations 25
14.2 Future work 25
15. Conclusion 26
References 27
Appendix A. Repository Structure 28
Appendix B. Reproducing the Results 28
Appendix C. Glossary 28

1. Introduction

1.1 Background and motivation

Lectures, group meetings and interviews produce spoken content that is hard to search. A recording can be played back, but finding one remark means listening to the whole file. Converting speech to text makes recordings searchable and reviewable, and it supports note-taking and accessibility for listeners who are deaf or hard of hearing. The conversion is not perfect. Recognition quality depends on accent, background noise, speaking speed and language, so a transcript must be checked before it is relied on.

1.2 Problem statement

The project brief (AAI-010, Speech AI) asks for a system that converts a short recorded speech clip into text using a pretrained speech-recognition library or API. It requires the team to use short clips with clear speech, keep a manually checked transcript as a reference, and evaluate the output. The required workflow is to load an audio file, run pretrained recognition, clean the transcript formatting, and compare the transcript with the reference. The deliverable must show the clip details, the generated transcript, corrections, and a basic qualitative error review or word-error estimate. The brief also asks the team to record the model and settings used, and to handle recorded speech responsibly.

1.3 Objectives

- Transcribe short English clips locally with a pretrained model, without training a new model.
- Clean the raw output into readable lines with consistent capitalisation and punctuation.
- Measure accuracy with Word Error Rate against a manually checked reference, and list every mismatch.
- Provide a web application that a non-technical user can operate, and a command-line pipeline that produces a reproducible report.
- Document the research, architecture, evaluation and deployment so that the work can be reproduced and extended.

1.4 Scope and contributions

The scope is English read speech in short clips of 20 to 40 seconds. The main contributions are: (i) a working end-to-end prototype; (ii) a WER implementation with a per-error report; (iii) a documented error analysis of two clips; (iv) a reproducible workflow for building clips and references from the LibriSpeech corpus; and (v) a deployment that runs free on Hugging Face Spaces. Out of scope are live microphone streaming, speaker diarisation, and fine-tuning on meeting audio.

2. Research and Background

2.1 Automatic speech recognition: a short history

Automatic speech recognition (ASR) maps an acoustic signal to a sequence of words. For several decades, the dominant approach was the hybrid of Gaussian mixture models (GMMs) for acoustic modelling and hidden Markov models (HMMs) for temporal structure, combined with an n-gram language model. Deep neural networks replaced the GMM emission model in the early 2010s, which reduced error rates substantially (Hinton et al., 2012). Connectionist temporal classification (CTC) allowed networks to be trained on unsegmented audio with transcripts as the only supervision (Graves et al., 2006). Attention-based encoder-decoder models, such as Listen, Attend and Spell, learned the alignment between audio and text directly (Chan et al., 2016). The transformer architecture (Vaswani et al., 2017) then became the standard building block for both encoders and decoders. Self-supervised pretraining on unlabelled audio, as in wav2vec 2.0, reduced the amount of labelled data needed (Baevski et al., 2020).

Whisper (Radford et al., 2022) took a different route. Instead of carefully curated data, it was trained with weak supervision on about 680,000 hours of audio collected from the web, with transcripts that were often imperfect. The authors reported that this scale makes the model robust to accents, background noise and technical language without task-specific fine-tuning. For a student project, this matters because a pretrained model can be used directly, and the effort goes into evaluation and engineering rather than training.

2.2 Choosing a recogniser for this project

The brief allows a pretrained library or an API. We compared the main options on the criteria that matter for a student prototype: cost, privacy, setup effort, accuracy and licence.

Option	Cost	Privacy	Setup	Notes
Cloud speech APIs (e.g. Google, Azure, AWS)	Free tier, then per minute	Audio leaves the machine	API keys and billing	Strong accuracy, but data handling depends on the provider
Vosk (Kaldi-based, offline)	Free	Local	Small models, lower accuracy	Good for constrained vocabularies
openai-whisper (chosen)	Free, MIT licence	Local or on the Space	pip install and ffmpeg	Robust across accents and noise; several model sizes
faster-whisper (CTranslate2)	Free	Local	pip install	Faster inference; same model family

Table 1. Comparison of recognition options considered for the project.

We chose openai-whisper because it is open source, runs without an API key, keeps audio local when run on a personal computer, and has documented model sizes. The base model gives a reasonable trade-off between accuracy and speed on a CPU.

2.3 Measuring recognition accuracy

The standard metric for ASR is word error rate. It is computed from a minimum-cost alignment between the reference and the hypothesis, using the Levenshtein edit distance (Levenshtein, 1966). Each reference word is either matched, substituted, or deleted, and each extra hypothesis

word is an insertion. WER is the number of errors divided by the number of reference words. Morris, Maier and Green (2004) discuss the limitations of WER and propose related measures, including the match error rate and the word information lost, which weigh insertions, deletions and substitutions differently. We report WER with its S, D and I breakdown so the reader can see which error type dominates.

2.4 Datasets

LibriSpeech (Panayotov et al., 2015) provides about 1,000 hours of read English from public-domain audiobooks, with aligned transcripts. Its development subsets are commonly used for evaluation. We used utterances from the dev-clean subset because they are clean, carry official transcripts, and are licensed under CC BY 4.0. Mozilla's Common Voice (Ardila et al., 2020) is an alternative with crowd-recorded and more varied audio; it was not used in this project, but it would be a good test of robustness to noise.

2.5 Responsible use of speech data

Recorded speech can identify people and reveal private conversations. The brief asks students to use consented or public audio, to test in one language, and to follow the terms of the tools they use. Our design follows these points: the model runs on the machine or Space that processes the audio, the web application does not write uploaded audio to disk, and the documentation warns against unconsented recordings. Transcripts can contain errors that change meaning, so the system presents accuracy figures and an error list rather than presenting the transcript as final.

3. Requirements Analysis

3.1 Functional requirements

ID	Requirement	Met by
FR1	Accept a short audio file (wav, mp3, m4a, ogg, flac) or a microphone recording	app.py audio input; data/clips for batch use
FR2	Transcribe speech into text with a pretrained model	src/transcribe.py; transcribe_audio() in app.py
FR3	Clean spacing, punctuation, capitals and line breaks	src/cleanup.py
FR4	Compare with a reference transcript and report WER	src/wer.py; src/evaluate.py; run()
FR5	List every word-level error	error_lines() and the Word-level errors box
FR6	Let the user download the transcript	gr.File output
FR7	Keep a session history of results	HISTORY list and History tab
FR8	Record model and settings with each transcript	settings field in JSON output; model size in the UI
FR9	Build a PDF report from results	report/make_report.py; run_pipeline.py

Table 2. Functional requirements.

3.2 Non-functional requirements

- **Cost:** the public deployment must run on a free tier (Hugging Face Spaces with ZeroGPU).
- **Reproducibility:** the software versions and model size must be recorded, and the pipeline must run from one command.
- **Privacy:** uploaded audio is processed on the Space's server and is not stored by the application code.
- **Usability:** the interface must be usable without knowledge of Python, with clear steps and error messages.
- **Portability:** the same code must run on Windows, macOS, Linux and in a container.
- **Maintainability:** the metric, clean-up and recognition code is kept in separate modules with unit tests.

3.3 Data requirements

Each evaluation clip needs an audio file of 20 to 40 seconds with clear speech, a single speaker, and one language, together with a reference transcript that a person has checked against the audio. The reference must use the same word forms as the spoken text, because the metric counts spelling differences as errors.

4. System Architecture

4.1 Overview

The system has four layers: an interface layer (the browser UI or the command line), an application layer (the Gradio event handler or the pipeline scripts), a processing layer (audio decoding, Whisper recognition and text clean-up), and an evaluation layer (WER computation and reporting). Figure 1 shows how these layers connect for the web application.

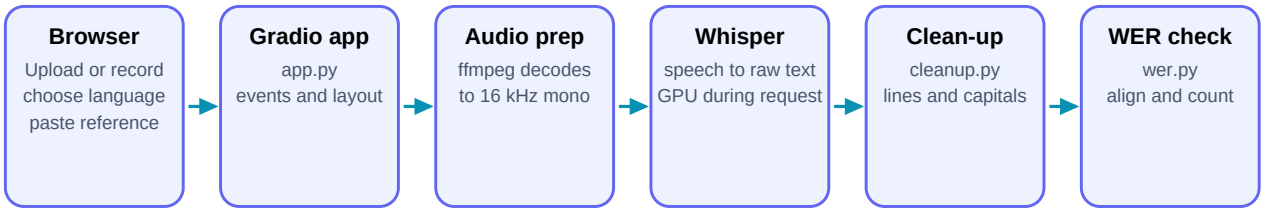


Figure 1. Architecture of the web application, from the browser to the WER check and the results panel.

4.2 Components

Component	File	Responsibility	Key functions
Web interface	app.py (UI)	Layout, theme, tabs, events	gr.Blocks, gr.Audio, gr.Textbox, gr.Dataframe
Request handler	app.py	Validate input, call model, compute WER, build outputs	run()
Model wrapper	app.py	Load Whisper once; move to GPU for the request	get_model(), transcribe_audio()
ffmpeg check	src/check_env.py	Find ffmpeg or use the bundled copy	check()
Clean-up	src/cleanup.py	Tidy spacing, punctuation and lines	clean_transcript()
Metric	src/wer.py	Normalise, align, count S, D, I; list errors	compute_wer(), align(), error_lines()
Batch transcription	src/transcribe.py	Run Whisper on a folder; save text and JSON	main()
Batch evaluation	src/evaluate.py	Write metrics.csv and error_review.md	evaluate()
Clip builder	src/make_clip.py	Join LibriSpeech utterances and write the reference	main()
Report	report/make_report.py	Build the PDF from the results	build()

Table 3. Components and their responsibilities.

4.3 Design decisions

- **One core, two front ends.** The web app and the command-line pipeline import the same functions in src/. A fix to the metric or the clean-up applies to both.
- **Load once, reuse.** Whisper is loaded on the first request and kept in memory, because loading takes longer than transcribing a short clip.

- **CPU load, GPU use.** The model is loaded on the CPU and moved to the GPU only inside the function decorated with @spaces.GPU. This is the pattern ZeroGPU expects, and the same code falls back to the CPU locally.
- **Bundled ffmpeg.** Whisper calls an ffmpeg executable. The imageio-ffmpeg package supplies one through pip, so the project needs no system package on Windows.
- **In-memory history.** The session history is kept in memory. Persistent storage would need a paid Space or an external database, which we did not use because the brief does not require it.

4.4 Evolution of the architecture

The first version was a React frontend with a FastAPI backend, served by a single Docker container. It worked locally and was tested with the team. Hugging Face Spaces offers Docker only on paid plans, so we rebuilt the interface in Gradio, which runs on the free tier. The transcription and WER code was kept unchanged, so the rebuild only replaced the presentation layer. This change is a concrete example of how a modular design reduces the cost of changing the front end.

5. The Recognition Model

5.1 Whisper in brief

Whisper is an encoder-decoder transformer. It receives 30-second windows of audio, converts each window into a spectrogram, encodes the spectrogram into a sequence of hidden states, and decodes text one token at a time while attending to the encoder states (Radford et al., 2022). The same network handles transcription, translation and language identification, controlled by special tokens at the start of the decoder sequence.

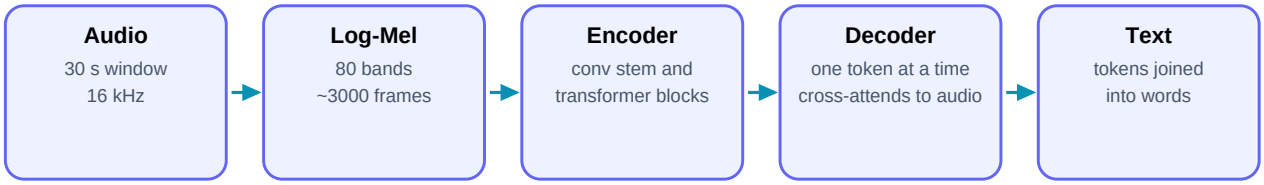


Figure 2. Whisper processing pipeline for one 30-second window.

5.2 Stages in detail

Resampling and windowing. Audio is converted to 16 kHz mono. Clips shorter than 30 seconds are padded with silence; longer clips are split into windows and transcribed in sequence.

Log-Mel spectrogram. The signal is cut into overlapping frames, a short-time Fourier transform is applied, and the energies are mapped onto 80 mel-spaced frequency bands on a log scale. The 25 ms window with a 10 ms hop gives 3000 frames for 30 seconds. The mel scale follows human hearing, which gives more resolution to low frequencies where most speech information lies.

Encoder. Two convolutional layers process the spectrogram, and the second halves the time resolution, giving 1500 positions. Sinusoidal position information is added, and a stack of transformer blocks with self-attention builds a contextual representation of the whole window.

Decoder. The decoder is a transformer that generates tokens from a byte-level BPE vocabulary of about 51,000 entries. Each step attends to the previously generated tokens and to the encoder states. The start of the sequence contains a token that marks the start of transcript, a language token (or a token predicted by the model when language detection is used), and a task token that selects transcription or translation. Timestamp tokens can be emitted between words.

Decoding. The openai-whisper implementation decodes greedily by default, and retries with a higher sampling temperature when the output appears repetitive or has low confidence. This behaviour is part of the library and was not changed in this project.

5.3 Model sizes

Model	Parameters (approx.)	Relative speed	Use in this project
tiny	39 M	Fastest	Fallback on very weak hardware
base	74 M	Fast	Used for all reported results and the deployed app
small	244 M	Medium	Candidate for comparison in future work
medium	769 M	Slow	Not used; too large for the free tier
large	1550 M	Slowest	Not used

Table 4. Whisper model sizes. Parameter counts are the published approximations.

5.4 Settings recorded with every transcript

Each transcript is saved with the Whisper package version (20250625 for the runs reported here), the model size (base), the language setting (en, or auto), the engine name, and whether fp16 was used. These settings make the results reproducible and let a reader see which model produced a transcript.

6. Implementation

6.1 Text clean-up (src/cleanup.py)

The clean-up function collapses whitespace, removes spaces before punctuation, and places each sentence on its own line. Abbreviations such as Mr., Mrs., Dr., St. and Prof. are protected from sentence splitting, which fixed a defect where "Mr. Quilter" was broken across two lines. The function also capitalises each line and adds a final full stop when one is missing. It does not change word forms, so it cannot change the WER.

6.2 Word error rate (src/wer.py)

The metric normalises both texts by lower-casing them, removing punctuation except apostrophes, and expanding the abbreviations Mr, Mrs, Ms and Dr to Mister, Missus, Miss and Doctor. The expansion ensures that a written abbreviation in Whisper output is not counted as an error when the reference spells the word out. The two word lists are then aligned with dynamic programming:

```
dp[i][j] = min( dp[i-1][j] + 1,          # deletion
                dp[i][j-1] + 1,          # insertion
                dp[i-1][j-1] + cost )    # match or substitution
cost = 0 if ref[i-1] == hyp[j-1] else 1
```

A backtrace from the bottom-right cell recovers which operation produced each step, so every substitution, deletion and insertion can be listed. WER is the number of errors divided by the number of reference words. When the reference is empty, WER is zero if the hypothesis is also empty and one otherwise. The module has no third-party dependencies, which makes it easy to inspect.

6.3 Recognition (src/transcribe.py and app.py)

The command-line transcriber loads Whisper once, runs it on each clip in the data folder, and writes a cleaned text file and a JSON file with the raw output and settings. The web app uses the same model, but calls it inside a function decorated with the ZeroGPU decorator. This decorator attaches a GPU for the duration of the call on Hugging Face Spaces. When the decorator is absent, as on a personal computer, the function runs on the CPU. Model weights are loaded on the CPU, moved to the GPU for the call, and the precision is set to fp16 only on the GPU.

6.4 Evaluation and reporting (src/evaluate.py and report/make_report.py)

The evaluation script pairs each reference file with the transcript of the same name, computes WER, and writes a CSV file of counts and a Markdown file listing every mismatch. The report script reads the CSV and builds the PDF, so the report is regenerated from the data each time the pipeline runs.

6.5 Clip construction (src/make_clip.py)

LibriSpeech stores each utterance as a FLAC file, and each chapter has a transcript file with one line per utterance. The clip builder takes the requested utterance identifiers, concatenates the FLAC files with ffmpeg into one mono 16 kHz WAV file, and writes the matching transcript (in lower case) as the reference. Because the reference comes from the official corpus, a student who wants a fully independent benchmark must listen to the clip and correct the reference by hand. We did this check for the reported clips.

6.6 Packaging and dependencies

Package	Purpose	Version used
openai-whisper	Pretrained recognition model	20250625
gradio	Web interface and hosting	5.50.0
imageio-ffmpeg	Bundled ffmpeg binary	0.7.0
reportlab	PDF report generation	5.x
spaces (preinstalled on ZeroGPU)	@spaces.GPU decorator	Provided by Hugging Face
ffmpeg (packages.txt)	Fallback system package on Spaces	System version

Table 5. Main dependencies and versions.

6.7 Code quality and tests

The metric and clean-up modules are covered by eleven unit tests, which check identical texts, single substitutions, deletions and insertions, case and punctuation handling, apostrophes, WER values above one, an empty reference, and the clean-up rules. All eleven tests passed. The tests run without a speech model, so they can run on any machine. The web interface and the Whisper integration were checked by running the application locally and through the Space's build logs, because they require a model and a browser.

7. Experimental Setup

7.1 Data

Two clips were built from LibriSpeech dev-clean. Clip 1 uses three consecutive utterances from one chapter and contains 59 reference words. It is a read passage with ordinary vocabulary and a small number of names. Clip 2 uses a different set of utterances from the same corpus and contains 110 reference words, including many proper names and older, literary vocabulary. A third clip is in preparation and is not reported here.

7.2 Hardware and software

Environment	Hardware	Software
Local development	Windows 11 laptop, CPU	Python 3.11, openai-whisper 20250625, Gradio 5.50.0
Hugging Face Space	ZeroGPU (shared GPU, allocated per request)	Python 3.10 container, Gradio 5.50.0, ffmpeg

Table 6. Experimental environments.

7.3 Procedure

- Build each clip with src/make_clip.py, which records the utterance identifiers and the source corpus.
- Check the reference against the audio by listening, and correct any word that differs.
- Transcribe with the base model, language English, using the command-line pipeline.
- Clean the transcript with clean_transcript() and score it with compute_wer().
- Record the counts of substitutions, deletions and insertions, and list every mismatch.
- Repeat the scoring after each change to the normalisation, and record both results where the change affects the score.

7.4 Procedure for the abbreviation change

The first scoring of clip 1 gave 6.78% WER. Two of the three substitutions were Mr against Mister, which are the same word written differently. We expanded common abbreviations in the normalisation step and re-scored the same transcript, without changing the model or the reference. The result fell to 3.39%. We report both values, because the change to the metric is part of the analysis. The change makes the score more accurate for the purpose of measuring recognition, but it does not hide any real error.

8. Evaluation Method

8.1 Error types

Error	Definition	Example from clip 2
Substitution (S)	A reference word replaced by a different hypothesis word	"Leighton's" written as "Layton's"
Deletion (D)	A reference word missing from the hypothesis	"up" missing before "guards"
Insertion (I)	A hypothesis word not present in the reference	None in clip 2; one in clip 1 ("similarly")

Table 7. Definitions of the three error types.

8.2 Qualitative review

WER does not show why errors occur. For each mismatch we assigned one category: proper name, rare or archaic word, word boundary (a word split or merged), function word, or other. The categories were chosen after reading the error lists, so they describe these two clips rather than a general taxonomy. Each error was also checked against the audio when the cause was unclear.

8.3 Limits of the metric

- WER weights every word equally. A wrong name and a wrong "the" count the same, although they affect the meaning differently.
- Spelling variants of names are scored as errors even when the sound is correct.
- A short clip gives a noisy estimate. One error more or less changes WER by about one percentage point on a 110-word clip.
- The reference is one human transcript. Another annotator might spell names or split words differently, which would change the score.

9. Results

9.1 Summary

Run	Reference words	Substitutions	Deletions	Insertions	WER
Clip 1, before abbreviation fix	59	3	0	1	6.78%
Clip 1, after abbreviation fix	59	1	0	1	3.39%
Clip 2	110	9	2	0	10.00%
Clip 3	pending	-	-	-	-

Table 8. Word error rate for each clip and run.

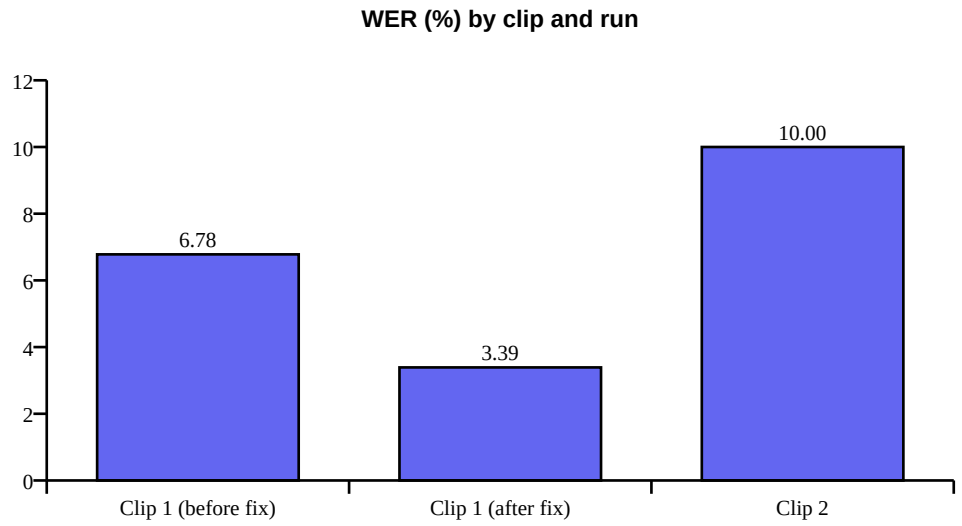


Figure 3. WER for each clip and run. Clip 2 is more than twice as hard for the model as clip 1.

9.2 Interpretation

Clip 1 is a passage with few names and common vocabulary, and the model made one substitution and one insertion in 59 words. Clip 2 contains older vocabulary and several names, and the error rate is three times higher. The difference shows that a single headline figure hides the content of the audio. A reader should report the content of each clip next to its WER.

9.3 Clip-level statistics

With 110 reference words, the 95% confidence interval for a 10% error rate is wide. A normal approximation to the binomial gives a half-width of about 5.6 percentage points, so the true rate on this kind of text could lie anywhere from roughly 4% to 16%. Two clips are not enough to estimate accuracy for general speech; they are enough to show the method and the failure patterns.

10. Error Analysis

10.1 Clip 2: all 11 errors

Clip 2 has 11 errors: nine substitutions and two deletions. Table 9 lists each error with its category and a likely cause. The output comparison shows the same errors in context.

#	Reference	Hypothesis	Type	Category	Likely cause
1	grave	graved	S	Rare or archaic word	Morphology of an unusual word form
2	Leighton's	Layton's	S	Proper name	Name spelling is not recoverable from sound
3	Linnell's	Lynelle's	S	Proper name	Uncommon surname; model chose a common spelling
4	up	(missing)	D	Word boundary	"up guards" run together with the next word
5	guards	upgards	S	Word boundary	Merged form of "up guards"
6	at	(missing)	D	Word boundary	"at em" recognised as one token
7	em	atom	S	Rare or archaic word	"em" is an informal form of "them"; model chose a real word
8	idylls	ittles	S	Rare or archaic word	Rare word with an unusual sound pattern
9	Birket	Birkett	S	Proper name	Name spelling; the reference spelling is less common
10	in	and	S	Function word	"in a Turkish bath" heard as "and a Turkish bath"
11	in	and	S	Function word	"how delicate in expression" heard as "delicate and expression"

Table 9. Clip 2 errors. S = substitution, D = deletion.

10.2 Summary by category

Category	Errors	Share of total
Proper names	3	27%
Word boundaries	4	36%
Function words	2	18%
Rare or archaic words	2	18%
Total	11	100%

Table 10. Clip 2 errors by category.

Word boundaries are the largest category, but they are not a sign of poor recognition of the words themselves. The two phrases "up guards" and "at em" are spoken quickly and merge into one sound. Proper names are the second largest category, and they are the errors most affected by the reference spelling. The two function-word errors may be real misrecognitions, and we verified both against the audio. Only the "in" to "and" errors change the grammatical structure of the sentence, so they are the most important for a reader.

10.3 Clip 1: the remaining errors

Clip 1 has three mismatches before the abbreviation fix: two Mr against Mister, and one extra word "similarly" with the substitution "similes" written as "he's". After the fix, the abbreviation mismatches disappear, which leaves one substitution and one insertion. The substitution and insertion are the same mis-hearing: the audio contains "similes drawn from eating", and the model wrote "similarly he's drawn from eating". The error is a real misrecognition, not a formatting issue.

10.4 What the errors tell us

- The model is accurate on common words and on sentence structure. Most of the errors are on names, rare words, and places where speech runs together.
- Formatting errors can inflate WER. The abbreviation issue increased the score for clip 1 by 3.39 points, so normalisation must be checked before results are compared.
- Reference quality matters. Two of the errors need a careful listen to decide whether the model or the reference is wrong.
- Clip difficulty varies. Literary text with names is harder than a modern passage, so results should always include the content type.

10.5 Qualitative examples

The clip 2 output reads as a sensible transcript even where the WER is 10%. The sentence "Mr. Birkett Foster's landscapes smile at one much in the same way that Mr. Carker used to flash his teeth" is recognised almost exactly, and the model keeps the Mr. abbreviation, which the scoring now treats as a match. The clearest errors are the function-word errors in the sentence "Before he says like a shampooer in a Turkish bath", which the model wrote as "and a Turkish bath". A reader who checks only the sense would miss this change; the WER report shows it.

11. End-to-End Walkthrough: Backend to Frontend

This section follows one request through the system, from the browser to the results panel. It is the basis for the diagrams in the presentation and for the code review.

11.1 Page load

- The browser requests the Space URL. Gradio serves the page, which includes the hero section, two tabs and the CSS and theme from the app.
- The Python process has already imported app.py. Importing checks for ffmpeg, which may use the bundled copy, and creates the interface. The Whisper model is not loaded yet.

11.2 Upload and parameters

- The user uploads a file or records audio. Gradio stores the file in a temporary location and passes its path to the event handler.
- The user chooses the language and optionally pastes a reference. These values are sent with the request.

11.3 Transcribe click

Step	Where	Action
1	Browser	Gradio sends audio path, reference text and language to the server
2	run() in app.py	Checks that audio is present and that ffmpeg is available
3	transcribe_audio()	Runs under the ZeroGPU decorator; attaches a GPU if one is free
4	get_model()	Loads Whisper base on the first call, then reuses it
5	model.to(device)	Moves the model to CUDA when available, otherwise keeps it on the CPU
6	model.transcribe()	Decodes the audio, splits it into 30-second windows, and returns raw text
7	clean_transcript()	Produces the readable transcript
8	compute_wer()	Only when a reference is present: normalises, aligns, counts S, D, I
9	stat_cards()	Builds the coloured HTML cards for WER and error counts
10	error_lines()	Builds the list of mismatches for the error box
11	run() return	Writes the transcript to a .txt file and adds a row to the history

Table 11. Steps for one transcription request.

11.4 Results on screen

Gradio sends the returned values to the page: the transcript fills a text box with a copy button, the HTML cards replace the empty state, the error list fills its box, the download link appears, and the history table gains a row. Each value is bound to one output component, so the page updates without a full reload.

11.5 Error handling

Missing audio and missing ffmpeg raise a Gradio error, which the browser shows as a notification. Unsupported file types are rejected by the audio component. Empty references skip the WER step and show a hint instead. Errors inside Whisper stop the request, and Gradio returns the message

to the page.

11.6 Deployment path

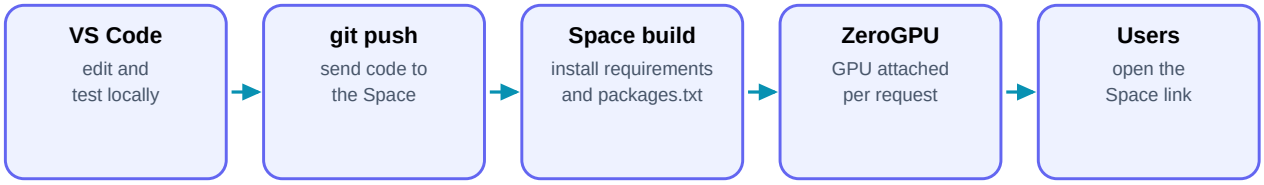


Figure 4. Deployment path from a developer's computer to a user's browser.

12. Development Log and Issues Resolved

The project was built in stages, and several problems came up on the way. Recording them is part of the method, because each one affected how the final system was designed.

Issue	Cause	Resolution
Transcript split at "Mr."	Sentence splitter treated every full stop as a boundary	Protected common abbreviations in cleanup.py
WER inflated by "Mr" vs "Mister"	Written abbreviation counted as a different word	Expanded abbreviations in wer.py normalisation
ffmpeg not found on Windows	ffmpeg is a system program, not a pip package	Bundled ffmpeg via imageio-ffmpeg; check_env.py copies it to .tools
Installed package conflicts	Gradio 6 installed while the code targeted Gradio 5	Pinned gradio>=5,<6 and recorded the version
Deprecation warnings	theme and css moved between Gradio versions	Passed them to gr.Blocks for Gradio 5.50.0
Space would not start	Free Spaces offer ZeroGPU, not CPU, and the code had no GPU function	Added @spaces.GPU to the transcription call
Docker Space unavailable	Docker SDK is paid on Hugging Face	Rebuilt the interface in Gradio
Torch installed the wrong build	CPU index URL installs a CPU-only PyTorch	Removed the CPU index line for the ZeroGPU Space

Table 12. Problems encountered and how they were resolved.

13. Responsible Use, Privacy and Licensing

13.1 Privacy

Speech carries personal information. The project uses only public-domain or consented audio, and the sample clips come from LibriSpeech. The web application does not write uploaded audio to a persistent store. On the Space, the audio is processed by the Space's server, and Hugging Face's own platform policies apply to the hosting. Users should not upload recordings of other people without their consent.

13.2 Accuracy and misuse

A transcript can look authoritative while containing errors that change meaning, as clip 2 shows with "in" becoming "and". The interface therefore presents the error count and the list of mismatches next to the transcript, and the documentation asks users to check transcripts before relying on them. The tool should not be used to make decisions about people, such as disciplinary or legal decisions, based on automatic transcripts.

13.3 Licences

Component	Licence	Use in this project
openai-whisper and model weights	MIT	Recognition; cited
LibriSpeech corpus	CC BY 4.0	Evaluation clips; cited
Gradio	Apache 2.0	Web interface
imageio-ffmpeg / ffmpeg	BSD / LGPL or GPL, depending on build	Audio decoding; not modified
reportlab	BSD	PDF report

Table 13. Licences of the main components.

The repository has no licence file yet. If the project is published for reuse, a licence such as MIT should be added to the repository, after confirming that it is compatible with every component above.

14. Limitations and Future Work

14.1 Limitations

- The evaluation uses two clips for reported results, so the accuracy estimate has wide uncertainty.
- All clips are read English. Accents, spontaneous speech, overlapping speakers and noise are not tested.
- The reference transcript is a single human version, and the spelling of names affects the score.
- Clip construction relies on LibriSpeech, so the clips are cleaner than typical meeting audio.
- The history is kept in memory and is lost when the Space restarts.
- ZeroGPU allocates a shared GPU with a daily quota, so heavy use may be slowed.

14.2 Future work

- Finish the evaluation on clip 3 and add at least five clips of different types.
- Compare the tiny, base and small models on the same clips, and report accuracy against model size and time.
- Add noisy and accented recordings, for example from Common Voice, to test robustness.
- Add speaker labels for meetings, using a diarisation model alongside the recogniser.
- Allow a reference file to be uploaded, and export the error review as a file.
- Store history in a persistent database so that results survive restarts.

15. Conclusion

This project delivered a working speech-to-text prototype that converts short clips into clean text, measures the accuracy against a human reference, and explains the errors. The system uses a pretrained model, so the effort went into the engineering and the evaluation. The main results are a 3.39% WER on a 59-word clip after an abbreviation fix in the scoring, and a 10.00% WER on a 110-word clip of literary text. The error analysis shows that the model handles common words and sentence structure well, and that its errors concentrate on names, rare words and merged words.

The development also showed that the measurement must be checked as carefully as the model. A formatting artefact inflated the first score, and the choice of normalisation changes the result. Reporting the content of each clip and the breakdown of errors makes the figures more useful than a single headline number.

References

Ardila, R., Branson, M., Davis, K., Henretty, M., Kohler, M., Meyer, J., Morais, R., Saunders, L., Tyers, F. M., & Weber, G. (2020). Common Voice: A massively-multilingual speech corpus. *Proceedings of LREC*.

Baevski, A., Zhou, H., Mohamed, A., & Auli, M. (2020). wav2vec 2.0: A framework for self-supervised learning of speech representations. *Advances in Neural Information Processing Systems*, 33.

Chan, W., Jaitly, N., Le, Q., & Vinyals, O. (2016). Listen, attend and spell: A neural network for large vocabulary conversational speech recognition. *Proceedings of ICASSP*.

Graves, A., Fernández, S., Gomez, F., & Schmidhuber, J. (2006). Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks. *Proceedings of ICML*.

Hinton, G., Deng, L., Yu, D., Dahl, G. E., Mohamed, A., Jaitly, N., Senior, A., Vanhoucke, V., Nguyen, P., Sainath, T. N., & Kingsbury, B. (2012). Deep neural networks for acoustic modeling in speech recognition. *IEEE Signal Processing Magazine*, 29(6).

Levenshtein, V. I. (1966). Binary codes capable of correcting deletions, insertions, and reversals. *Soviet Physics Doklady*, 10(8).

Morris, A. C., Maier, V., & Green, P. (2004). From WER and RIL to MER and WIL: Improved evaluation measures for connected speech recognition. *Proceedings of Interspeech*.

Panayotov, V., Chen, G., Povey, D., & Khudanpur, S. (2015). LibriSpeech: An ASR corpus based on public domain audio books. *Proceedings of ICASSP*.

Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., & Sutskever, I. (2022). Robust speech recognition via large-scale weak supervision. *arXiv:2212.04356*.

Abid, A., Abdalla, A., Abid, A., Khan, D., Alfozan, A., & Zou, J. (2019). Gradio: Hassle-free sharing and testing of ML models in the wild. *arXiv:1906.02569*.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, ■., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30.

Software and documentation: openai-whisper (github.com/openai/whisper, MIT licence); Gradio (gradio.app); Hugging Face Spaces (huggingface.co/docs/hub/spaces); ffmpeg (ffmpeg.org); imageio-ffmpeg (github.com/imageio/imageio-ffmpeg).

Appendix A. Repository Structure

```
speech-notes-prototype/
  app.py                 Gradio web application (entry point on Spaces)
  requirements.txt       Python packages
  packages.txt           system packages (ffmpeg)
  README.md              project documentation and Space header
  .gitignore             excludes audio, environments and results
  src/
    transcribe.py        batch transcription
    cleanup.py           transcript clean-up
    wer.py               WER and error alignment
    evaluate.py          batch scoring
    make_clip.py         clip and reference builder
    check_env.py         ffmpeg check and bundled copy
  tests/
    test_wer_and_cleanup.py   eleven unit tests
  report/
    make_report.py       PDF report builder
  run_pipeline.py        runs transcription, scoring and report
```

Appendix B. Reproducing the Results

- Install Python 3.11, then run: *pip install -r requirements.txt*.
- Place the clips in data/clips and the references in data/references, using matching names.
- Run *python run_pipeline.py* to transcribe, score and build the report.
- Run *python -m unittest discover -s tests -v* to check the metric and clean-up code.
- To use the web application, run *python app.py* and open the local address shown in the terminal.

Appendix C. Glossary

Term	Meaning
ASR	Automatic speech recognition
WER	Word error rate: (S + D + I) divided by the number of reference words
Log-Mel spectrogram	A time-frequency representation of audio on a log scale and a mel frequency axis
Encoder-decoder	A model that reads the input into hidden states and then generates the output
Token	A unit of text used by the model, often part of a word
ZeroGPU	Hugging Face's shared GPU allocation for free Spaces
Space	A Hugging Face repository that runs a web app